
Violyt / BrandLoveStudio.AI

System Architecture Document

Multi-tenant SaaS platform for brand-aware marketing creative generation
with LangGraph AI pipeline, Brand Space grounding, and premium image output.

Version 1.0 | 2026-07-29

Prepared for client review

Status: Current implementation (codebase-aligned)

1. Table of Contents

1. Table of Contents
2. Executive Summary
3. Product Purpose & Scope
4. High-Level System Architecture
5. Technology Stack
6. Deployment Topology (Docker)
7. Creative Intelligence Pipeline (L1–L10)
8. End-to-End Generation Flow
9. Brand Space, Knowledge & RAG
10. API Surface Overview
11. Security, Tenancy & Roles
12. External Integrations
13. Storage & Data Assets
14. Frontend Workspace
15. Non-Functional Considerations
16. Glossary

2. Executive Summary

Violyt (also referred to as BrandLoveStudio.AI) is a production-oriented, multi-tenant platform that turns brand knowledge and campaign briefs into finished social creatives. Tenants configure Brand Spaces (identity, voice, visuals, guardrails, knowledge). Users prompt the Studio workspace; a structured AI pipeline retrieves brand context, plans strategy and layout, drafts copy, prepares an approvable Creative Blueprint, then generates premium images with exact brand logo compositing.

The system is designed for fintech-grade brand fidelity (exemplified by Jiraaf creative DNA: ice-blue backgrounds, navy typography, orange accents, SEBI carousel footers where applicable) while remaining brand-configurable for other tenants.

- **Primary creative path:** LangGraph pipeline (L1→L7c approval → L8 image generation).
- **Secondary path:** ContentService / AI Orchestrator for broader content workflows.
- **Delivery:** FastAPI backend, Next.js frontend, PostgreSQL, Redis, Docker Compose.
- **Grounding:** Brand RAG (Pinecone namespaces), OCR on brand assets, live research for data creatives.

3. Product Purpose & Scope

3.1 In scope

- Multi-tenant auth, RBAC, usage limits, and tenant administration
- Brand Space lifecycle, section configuration, and asset upload
- Knowledge ingestion (OCR → chunk → embed → vector retrieve)
- Chat / Studio creative generation (static, carousel, infographic)
- Blueprint approval gate before image generation
- AI image generation with logo overlay and brand colour locks
- Jobs/workers, analytics scaffolding, review links, export helpers

3.2 Out of scope / scaffolding

- Full live social network publishing (connect/publish scaffolding only)
- Complete L10 quality evaluation (currently a pass-through stub in default runs)
- Third-party Brand Space marketplaces

4. High-Level System Architecture

Clients interact with a Next.js workspace. All business logic and AI orchestration run on the FastAPI backend. PostgreSQL is the system of record; object storage holds uploads and generated assets; a vector store grounds generation in brand documents.

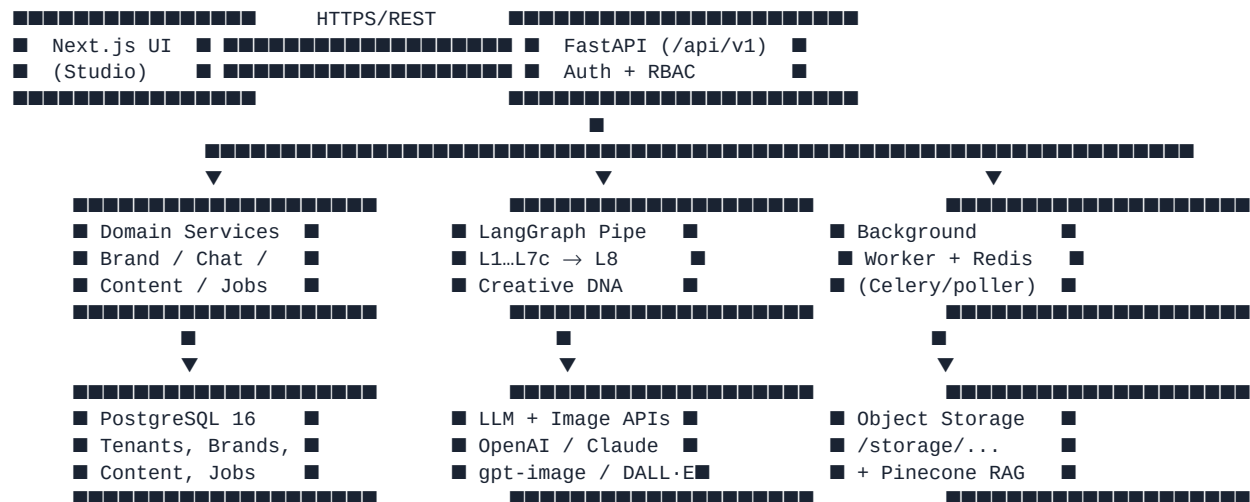


Figure 1 — Logical system context

4.1 Layered backend design

Layer	Responsibility	Key locations
API	Versioned REST, validation, HTTP errors	main.py, app/api/
Auth	JWT, roles, tenant/brand scope	app/core/, app/services/auth.py
Services	Business logic & orchestration	app/services/
Graph	Creative pipeline L1–L10	app/graph/
Prompts	Layer prompts + brand DNA locks	app/prompts/
Repos	SQLAlchemy models & repositories	app/models/, app/repositories/
Integrations	Storage, vectors, providers	app/integrations/, app/ai/
Workers	Async ingestion & jobs	app/workers/, scripts/

5. Technology Stack

Area	Technology
Backend	Python 3.12+, FastAPI, Uvicorn, SQLAlchemy async, Alembic, Pydantic
Frontend	Next.js 16, React 19, TypeScript, Tailwind, TanStack Query
Database	PostgreSQL 16
Cache / broker	Redis 7; Celery broker; Postgres job poller
Orchestration	LangGraph state machine
LLMs	OpenAI + Anthropic Claude (routed per pipeline layer)
Image generation	OpenAI gpt-image / DALL·E path; SDXL service stub
RAG / embeddings	OpenAI embeddings; Pinecone namespaces; FAISS abstraction
OCR / Vision	Google Cloud Vision + local OCR processors
Rendering aids	Pillow compositing (logo, SEBI footer on carousels)
Email	SMTP transactional mail

6. Deployment Topology (Docker)

Local and self-hosted stacks are defined in **docker-compose.yml** (and server variant under **deploy/**).

Service	Role	Host port
api	FastAPI; migrations then Uvicorn; mounts storage	8000
worker	Background jobs (scripts/run_worker.py)	—
violyt-frontend	Next.js workspace UI	3001 → 3000
postgres	Primary database	5432
redis	Cache + Celery broker	6379

Note: Backend images are built locally as **violyt-backend:local** / **violyt-frontend:local** — they are not pulled from Docker Hub. API health is exposed at **GET /health**. Generated files are served under **/storage/...**

7. Creative Intelligence Pipeline (L1–L10)

The primary Studio path uses a two-phase LangGraph. Phase 1 produces an approvable Creative Blueprint. Phase 2 generates images only after explicit approval, reducing wasted image cost and improving brand control.

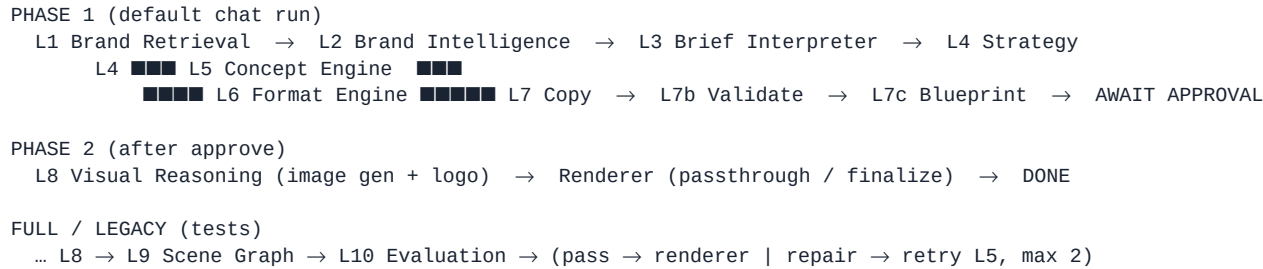


Figure 2 — Pipeline phases

Layer	Function
L1 Brand Retrieval	Namespace-isolated RAG; brand document grounding; retrieval logs
L2 Brand Intelligence	Structured brand behaviour model; Redis-cached by brand version
L3 Brief Interpreter	Maps user prompt + platform/format into campaign brief
L4 Strategic Reasoning	Angles, narrative strategy from brand + brief
L5 Concept Engine	Multiple creative concepts with diversity controls
L6 Format Engine	Layout/slide plan for static, carousel, infographic (parallel with L5)
L7 Copy Engine	Final copy; optional live research; layout classification
L7b Content Validator	Spell/length/fact flags; image-safe string limits
L7c Content Prep	Creative Blueprint for human approval + quality finalize
L8 Visual Reasoning	Image prompts + gpt-image/DALL-E; brand colour/fit locks; logo
L9 Scene Graph	Structured scene zones (full graph)
L10 Evaluation	Quality gate (stub pass in default path)
Repair / Renderer	Retry loop to L5; Phase 2 image URL finalize

7.1 LLM routing (typical)

- **Anthropic Claude:** L2, L4, L5 (brand intelligence, strategy, concepts).

- **OpenAI:** L3, L6, L7, L7c, L8, L9 (brief, format, copy, blueprint, visuals).
- **Image:** OpenAI image models via DalleService with Pillow logo/SEBI composites.

7.2 Brand creative DNA

Prompt locks in **app/prompts/brand_copy_tone.py** and layout routers in **jiraaf_layout.py** enforce sample-aligned layouts: ice-blue backgrounds, navy headlines, orange accents ($\geq \sim 2\%$ coverage), universal fit margins, topic-specific 3D icons, and carousel-only SEBI footers when required.

8. End-to-End Generation Flow

- User enters a prompt in Studio (platform + format selected).
- **POST /api/v1/pipeline/run** — intent/layout may override format for data boards.
- Phase 1 graph runs L1→L7c; checkpoint saved; status **awaiting_blueprint_approval**.
- User reviews/edits blueprint → **POST /api/v1/pipeline/approve** (or reject).
- Phase 2 L8 generates image(s); logo composited; assets stored under tenant/brand paths.
- Frontend displays asset URLs; optional text edit endpoint for post-fixes.

Alternate path: **POST /api/v1/content/generate** uses ContentService + AIOrchestrator for planning, RAG, render, and history — still available for broader content workflows.

9. Brand Space, Knowledge & RAG

Brand Spaces are first-class tenant resources with lifecycle **draft** → **active** → **archived/deleted**. Ten section domains cover identity, foundations, voice, personas, guardrails, knowledge, prompt intelligence, objectives, visual identity, and review.

- Uploads → OCR/vision analysis → chunking → embeddings → Pinecone namespace **brand:{id}**.
- L1 retrieval is tenant/brand isolated and logged for auditability.
- Resolved brand context + snapshots feed downstream layers after validation.
- Logos and visual assets are stored for deterministic overlay on generated creatives.

10. API Surface Overview

Base path: **/api/v1**. Interactive docs: **/docs**.

Group	Purpose
/auth	Login, refresh, profile, 2FA, password reset
/tenants	Tenant admin, users, usage limits
/brands	Brand Space lifecycle, sections, retrieval
/knowledge	Knowledge upload / reprocess
/chat	Sessions and messages
/pipeline	LangGraph run / approve / reject / edit
/content	Generate, rewrite, history, export
/templates	Upload, recommend, apply
/render	Layout preview / export helpers
/review	External share & comment links
/analytics	Platform / tenant / brand usage
/jobs	Background job status
/social	Connector scaffolding

11. Security, Tenancy & Roles

- JWT bearer authentication; refresh tokens; optional 2FA.
- Roles: super_admin, tenant_admin, tenant_user, brand_user, external_reviewer.
- Data isolation by **tenant_id** and **brand_space_id**.
- Super Admin blocked from Brand Space content generation flows.
- Usage limits gate expensive AI actions.
- Social tokens encrypted at rest; secrets via environment configuration.

12. External Integrations

Integration	Use
OpenAI	Text layers, embeddings, research, image generation
Anthropic Claude	Brand intelligence, strategy, concepts
Pinecone	Brand document vector index (namespaced)
Google Cloud Vision	OCR / document vision
SMTP	Transactional email
Redis	L2 brand cache + Celery broker
Brave Search (optional)	Alternate live research provider

13. Storage & Data Assets

- Object storage (local default): `{tenant}/{brand}/{category}/...` under `OBJECT_STORAGE_BASE_PATH`.
- Categories include uploads, generated images, logos, templates, pipeline checkpoints, traces.
- PostgreSQL stores tenants, brands, chat, content history, jobs, analytics events.
- Pipeline checkpoints enable resume after blueprint approval.

14. Frontend Workspace

The Next.js app under **frontend/** provides the Studio chat workspace, Brand Space administration surfaces, and pipeline approval UX. It talks to the backend via typed contracts (**contracts/frontend-api.ts**) and hooks such as **usePipeline**. Live API mode and mock UI mode are both supported for design review.

15. Non-Functional Considerations

- **Cost control:** Blueprint approval before image generation; layer-specific model routing.
- **Brand safety:** Guardrails sections, copy validators, spelling/fit locks, logo overlay.
- **Observability:** Structured logs, retrieval logs, generation traces.
- **Scalability:** Stateless API containers; worker separation; Redis/Celery ready.
- **Portability:** Docker Compose for local; server compose for host deployments.
- **Extensibility:** Prompt builders and layout routers allow brand DNA upgrades without UI rewrites.

16. Glossary

Term	Meaning
Brand Space	Tenant-scoped brand configuration + knowledge container
Creative Blueprint	L7c structured plan approved before image spend
LangGraph	Stateful multi-step AI orchestration framework
RAG	Retrieval-Augmented Generation from brand documents
SEBI footer	Regulatory disclaimer composited on Jiraaf carousels only
Layout DNA	Prompt locks that force sample-aligned visual structure

This document reflects the implemented architecture of the Violyt / BrandLoveStudio.AI codebase. For operational runbooks, see internal docs: `ARCHITECTURE.md`, `CREATIVE_GENERATION_ARCHITECTURE.md`,

DOCKER_SETUP.md, and API_CONTRACTS.md.

— End of document —